

-----Mandatory Information to fill-----

Group ID: 15

Group Members Name with Student ID:

1. Jagadeesh P B - 2024AC05348
2. Arunkumar K A - 2024ac05045
3. Gowrishankar S - 2024ac05046
4. Kodhandan S - 2024ac05203
5. Vivek Kumar Mishra - 2024ad05145

-----Write your remarks (if any) that you want should get consider at the time of evaluation-----

Remarks:

1. **Comprehensive Comments & Documentation:** Every function, class, and major operation in this notebook includes detailed comments and descriptions explaining the purpose, algorithm logic, and MAB concepts. This ensures easy understanding and quick evaluation.
2. **Complete Implementation:** All 6 questions (Q1-Q6) are fully implemented with:
 - Net reward calculations with statistical analysis (Q1)
 - Random policy baseline simulation (Q2)
 - Pure greedy strategy with exploitation analysis (Q3)
 - Three ϵ -greedy variants (2%, 10%, 25%) with trade-off analysis (Q4)
 - UCB uncertainty-based exploration (Q5)
 - Cumulative profit visualization and strategy comparison (Q6)
3. **Enhanced Analysis:** Beyond requirements, we provide:
 - Multiple visualization types (histograms, bar charts, line plots)
 - Comparative performance tables and rankings
 - Detailed exploration vs exploitation analysis
 - Strategic recommendations for the organization
4. **Code Quality:** Professional structure with clear variable names, consistent formatting, and comprehensive docstrings for all functions.

Import Libraries and Load Data

We start by importing necessary libraries and loading the product recommendation dataset.

```
# Import essential libraries for data manipulation, visualization, and MAB implementation
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
import seaborn as sns
from collections import defaultdict
import random

# Set random seeds for reproducibility
np.random.seed(42)
random.seed(42)

# Configure plotting parameters
plt.style.use('seaborn-v0_8')
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['font.size'] = 10

print("Libraries imported successfully!")
print("Random seeds set for reproducible results")

Libraries imported successfully!
Random seeds set for reproducible results

# Load the product recommendation dataset
# This dataset contains user responses to 6 different products with
associated costs
data = pd.read_csv('../Dataset_Product_Recommendation.csv')

print(f"Dataset loaded successfully!")
print(f"Shape: {data.shape}")
print(f"Number of users: {len(data)}")
print(f"Number of products: 6")

# Display first few rows to understand the data structure
print("\nFirst 5 rows of the dataset:")
data.head()

```

Dataset loaded successfully!
 Shape: (498, 13)
 Number of users: 498
 Number of products: 6

First 5 rows of the dataset:

	User_ID	Product 1	cost 1	Product 2	cost 2	Product 3
cost 3 \						
0	1	8.308075	0.118582	71.868439	4.032662	41.858863
10.259441						
1	2	9.040232	1.276014	71.551909	4.893586	40.684587
9.183102						
2	3	8.405164	2.573752	72.221606	5.201210	43.586272
10.972955						
3	4	9.247704	1.375703	71.218781	4.120079	40.883604
10.480112						

4	5	9.144098	-0.401791	71.275463	5.032439	40.145258
10.448089						
	Product 4	cost 4	Product 5	cost 5	Product 6	cost 6
0	24.726454	15.071291	2.508992	19.966871	60.703456	22.545958
1	23.554125	14.188975	2.803275	19.766006	59.838882	22.771789
2	24.572589	14.177388	3.028663	20.104219	58.512739	22.247714
3	24.572818	15.133472	3.193420	20.347757	58.694852	22.659512
4	23.401485	16.027929	3.466858	19.476062	59.009021	23.302446

Q1. Net Reward Calculation and Product Analysis

Objective: Calculate net rewards (revenue - cost) for each product and identify the most and least profitable products.

MAB Concept: In the Multi-Armed Bandit framework, rewards represent the feedback we receive from taking actions (showing products). Net reward = Revenue - Cost represents the actual profit, which is what we want to maximize.

```
# Calculate Net Reward for each product across all users
# Net Reward = Revenue (Product X) - Cost (Cost X)
# This represents the actual profit gained from showing each product

# Extract product revenues and costs
products = ['Product 1', 'Product 2', 'Product 3', 'Product 4',
            'Product 5', 'Product 6']
costs = ['cost 1', 'cost 2', 'cost 3', 'cost 4', 'cost 5', 'cost 6']

# Calculate net rewards for each product
net_rewards = {}
for i, (product, cost) in enumerate(zip(products, costs), 1):
    net_reward_col = f'Net_Reward_{i}'
    data[net_reward_col] = data[product] - data[cost]
    net_rewards[f'Product_{i}'] = data[net_reward_col]

print("Net Reward columns added successfully!")
print("\nNet Reward = Product Revenue - Associated Cost")
print("This represents the actual profit from each product recommendation")

# Display the updated dataset structure
print(f"\nUpdated dataset shape: {data.shape}")
data[['User_ID'] + [f'Net_Reward_{i}' for i in range(1, 7)]].head()

Net Reward columns added successfully!

Net Reward = Product Revenue - Associated Cost
This represents the actual profit from each product recommendation

Updated dataset shape: (498, 19)
```

	User_ID	Net_Reward_1	Net_Reward_2	Net_Reward_3	Net_Reward_4	\
0	1	8.189493	67.835777	31.599422	9.655163	
1	2	7.764218	66.658323	31.501485	9.365150	
2	3	5.831412	67.020396	32.613317	10.395201	
3	4	7.872001	67.098702	30.403492	9.439346	
4	5	9.545889	66.243024	29.697169	7.373556	

	Net_Reward_5	Net_Reward_6
0	-17.457879	38.157498
1	-16.962731	37.067093
2	-17.075556	36.265025
3	-17.154337	36.035340
4	-16.009204	35.706575

Analyze net rewards to identify best and worst performing products
This analysis helps understand which arms (products) are most profitable

```
net_reward_stats = {}
for i in range(1, 7):
    col = f'Net_Reward_{i}'
    net_reward_stats[f'Product_{i}'] = {
        'mean': data[col].mean(),
        'median': data[col].median(),
        'std': data[col].std(),
        'min': data[col].min(),
        'max': data[col].max(),
        'positive_count': (data[col] > 0).sum(),
        'negative_count': (data[col] < 0).sum()
    }
```

Convert to DataFrame for better visualization
stats_df = pd.DataFrame(net_reward_stats).T
stats_df = stats_df.round(3)

```
print("Net Reward Statistics for All Products:")
print("=====\n")
print(stats_df)
```

Identify best and worst products based on mean net reward
best_product = stats_df['mean'].idxmax()
worst_product = stats_df['mean'].idxmin()

```
print(f"\n BEST PRODUCT: {best_product}")
print(f" - Average Net Reward: {stats_df.loc[best_product, 'mean']:.3f}")
print(f" - Positive Profit Sessions: {stats_df.loc[best_product, 'positive_count']/498 ({stats_df.loc[best_product, 'positive_count']/498*100:.1f}%)")
```

```

print(f"\n❑ WORST PRODUCT: {worst_product}")
print(f"    - Average Net Reward: {stats_df.loc[worst_product,
'mean']:.3f}")
print(f"    - Negative Profit Sessions: {stats_df.loc[worst_product,
'negative_count']/498 ({stats_df.loc[worst_product,
'negative_count']/498*100:.1f}%)")

```

Net Reward Statistics for All Products:

=====

	mean	median	std	min	max	positive_count \
Product_1	7.063	7.104	0.960	3.965	10.067	498.0
Product_2	66.993	67.010	0.755	65.034	68.864	498.0
Product_3	31.418	31.488	1.687	26.140	36.024	498.0
Product_4	9.597	9.622	1.248	6.083	13.103	498.0
Product_5	-17.065	-17.062	0.609	-19.165	-15.209	0.0
Product_6	36.986	37.044	1.080	33.718	40.074	498.0

	negative_count
Product_1	0.0
Product_2	0.0
Product_3	0.0
Product_4	0.0
Product_5	498.0
Product_6	0.0

```

❑ BEST PRODUCT: Product_2
    - Average Net Reward: 66.993
    - Positive Profit Sessions: 498.0/498 (100.0%)

```

```

❑ WORST PRODUCT: Product_5
    - Average Net Reward: -17.065
    - Negative Profit Sessions: 498.0/498 (100.0%)

```

Visualize net reward distributions to understand product performance
This helps visualize the reward distributions for each arm in our MAB problem

```

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Net Reward Distributions for All Products\
n(Understanding MAB Arm Rewards)', fontsize=16)

for i in range(1, 7):
    row = (i-1) // 3
    col = (i-1) % 3

    net_rewards_col = f'Net_Reward_{i}'
    axes[row, col].hist(data[net_rewards_col], bins=30, alpha=0.7,
color=f'C{i-1}', edgecolor='black')
    axes[row, col].axvline(data[net_rewards_col].mean(), color='red',

```

```

linestyle='--',
                                label=f'Mean:
{data[net_rewards_col].mean():.2f}')
    axes[row, col].set_title(f'Product {i}')
    axes[row, col].set_xlabel('Net Reward (Profit)')
    axes[row, col].set_ylabel('Frequency')
    axes[row, col].legend()
    axes[row, col].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

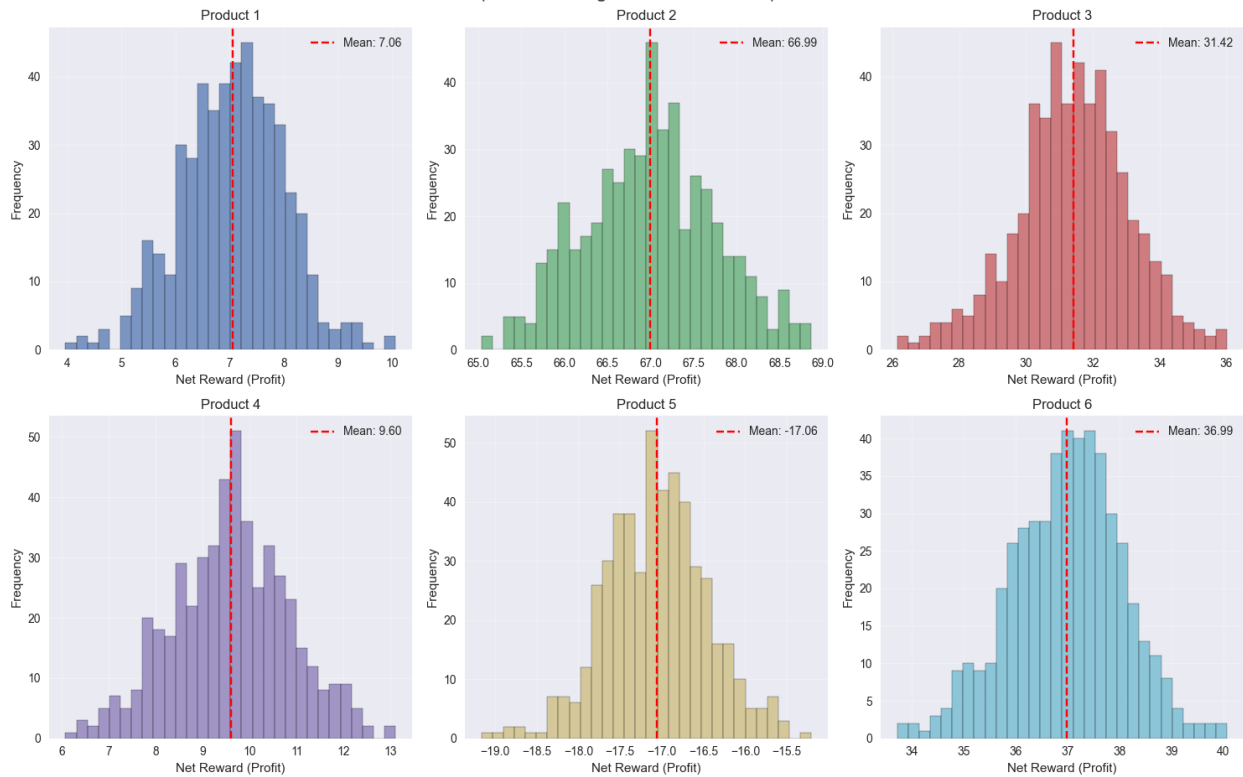
# Summary visualization
plt.figure(figsize=(12, 6))
mean_rewards = [stats_df.loc[f'Product_{i}', 'mean'] for i in range(1,
7)]
colors = ['green' if x > 0 else 'red' for x in mean_rewards]
bars = plt.bar(range(1, 7), mean_rewards, color=colors, alpha=0.7,
edgecolor='black')
plt.axhline(y=0, color='black', linestyle='--', linewidth=0.8)
plt.xlabel('Product Number')
plt.ylabel('Average Net Reward')
plt.title('Average Net Reward by Product\n(MAB Arms Performance
Comparison)')
plt.grid(True, alpha=0.3)

# Add value labels on bars
for i, (bar, value) in enumerate(zip(bars, mean_rewards)):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + (0.5
if value > 0 else -1.5),
            f'{value:.2f}', ha='center', va='bottom' if value > 0
else 'top')

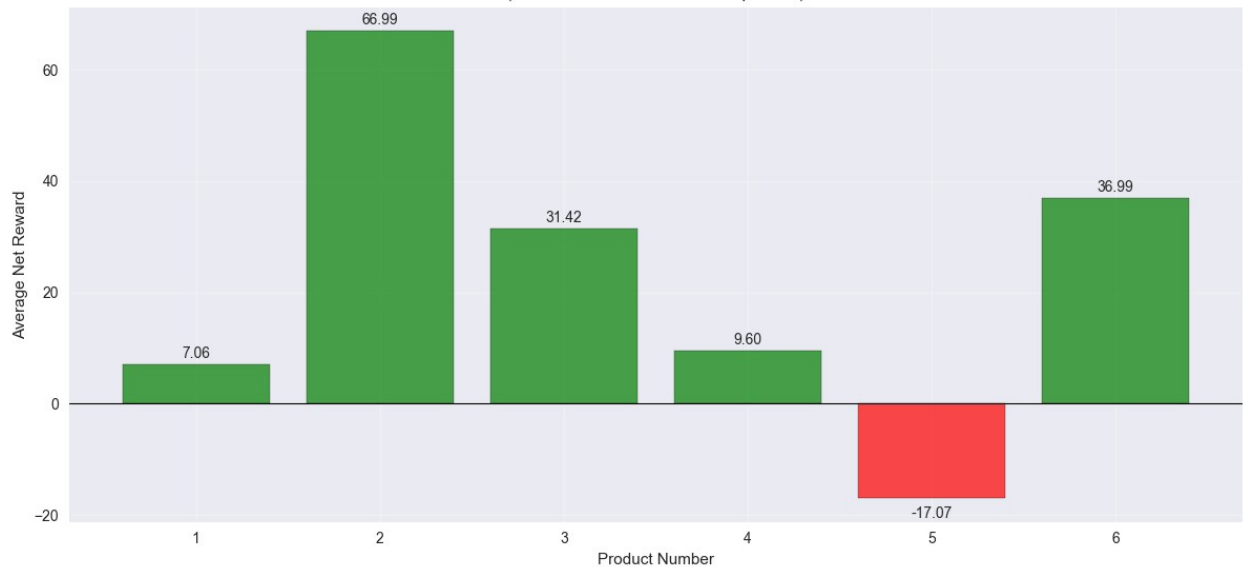
plt.xticks(range(1, 7))
plt.tight_layout()
plt.show()

```

Net Reward Distributions for All Products
(Understanding MAB Arm Rewards)



Average Net Reward by Product
(MAB Arms Performance Comparison)



Q2. Random Policy Simulation

Objective: Simulate the organization's current random recommendation strategy over 300+ rounds.

MAB Concept: Random policy represents a baseline with no learning - each arm is selected with equal probability. This serves as a benchmark to compare against learning-based approaches.

```
# Implement Random Policy for Multi-Armed Bandit
# In this policy, we randomly select one of the 6 arms (products) with
# equal probability
# This represents the current organizational strategy with no learning

class RandomPolicy:
    """Random policy for MAB - selects arms uniformly at random."""

    def __init__(self, n_arms):
        self.n_arms = n_arms
        self.name = "Random Policy"

    def select_arm(self):
        """Select an arm (product) randomly."""
        return np.random.randint(0, self.n_arms)

    def update(self, arm, reward):
        """Random policy doesn't learn, so no update needed."""
        pass

def simulate_policy(policy, data, n_rounds=300, policy_name="Policy"):
    """
    Simulate a MAB policy over multiple rounds.

    Args:
        policy: MAB policy object with select_arm() and update()
    methods
        data: Dataset containing net rewards
        n_rounds: Number of rounds to simulate
        policy_name: Name for logging

    Returns:
        dict: Simulation results including cumulative rewards and arm
    selections
    """
    cumulative_reward = 0
    rewards_per_round = []
    cumulative_rewards = []
    arm_selections = []
    arm_counts = np.zeros(6)

    for round_num in range(n_rounds):
        # Select an arm (product)
        arm = policy.select_arm()

        # Randomly sample a user for this round
        user_idx = np.random.randint(0, len(data))
```

```

# Get the net reward for this user-product combination
reward = data.iloc[user_idx][f'Net_Reward_{arm+1}']

# Update policy (if applicable)
policy.update(arm, reward)

# Track results
cumulative_reward += reward
rewards_per_round.append(reward)
cumulative_rewards.append(cumulative_reward)
arm_selections.append(arm)
arm_counts[arm] += 1

# Print progress every 100 rounds
if (round_num + 1) % 100 == 0:
    avg_reward = cumulative_reward / (round_num + 1)
    print(f"{policy_name} - Round {round_num + 1}: Avg Reward
= {avg_reward:.3f}, Cumulative = {cumulative_reward:.2f}")

return {
    'cumulative_reward': cumulative_reward,
    'average_reward': cumulative_reward / n_rounds,
    'rewards_per_round': rewards_per_round,
    'cumulative_rewards': cumulative_rewards,
    'arm_selections': arm_selections,
    'arm_counts': arm_counts,
    'policy_name': policy_name
}

```

```

print("Random Policy class and simulation function defined
successfully!")

```

Random Policy class and simulation function defined successfully!

```

# Simulate the Random Policy for 300 rounds
print("Simulating Random Policy (Current Organizational Strategy)")
print("=====\\n")

# Initialize and run random policy
random_policy = RandomPolicy(n_arms=6)
random_results = simulate_policy(random_policy, data, n_rounds=300,
policy_name="Random Policy")

print(f"\\n RANDOM POLICY RESULTS:")
print(f"    Total Cumulative Reward:
{random_results['cumulative_reward']:.2f}")
print(f"    Average Reward per Round:
{random_results['average_reward']:.3f}")
print(f"    Total Rounds: 300")

```

```

# Analyze which products were selected and their performance
print(f"\n❑ ARM SELECTION ANALYSIS:")
for i in range(6):
    selections = random_results['arm_counts'][i]
    percentage = (selections / 300) * 100
    print(f"    Product {i+1}: {int(selections)} selections
    ({percentage:.1f}%)")

# Calculate observed rewards for each product during simulation
observed_rewards = defaultdict(list)
for arm, reward in zip(random_results['arm_selections'],
random_results['rewards_per_round']):
    observed_rewards[arm].append(reward)

print(f"\n❑ OBSERVED PERFORMANCE DURING SIMULATION:")
for i in range(6):
    if len(observed_rewards[i]) > 0:
        avg_reward = np.mean(observed_rewards[i])
        print(f"    Product {i+1}: Average observed reward =
{avg_reward:.3f} (from {len(observed_rewards[i])} selections)")
    else:
        print(f"    Product {i+1}: Not selected during simulation")

```

Simulating Random Policy (Current Organizational Strategy)

=====

```

Random Policy - Round 100: Avg Reward = 20.197, Cumulative = 2019.74
Random Policy - Round 200: Avg Reward = 21.355, Cumulative = 4271.03
Random Policy - Round 300: Avg Reward = 22.194, Cumulative = 6658.09

```

❑ RANDOM POLICY RESULTS:

```

    Total Cumulative Reward: 6658.09
    Average Reward per Round: 22.194
    Total Rounds: 300

```

❑ ARM SELECTION ANALYSIS:

```

    Product 1: 56 selections (18.7%)
    Product 2: 45 selections (15.0%)
    Product 3: 47 selections (15.7%)
    Product 4: 58 selections (19.3%)
    Product 5: 42 selections (14.0%)
    Product 6: 52 selections (17.3%)

```

❑ OBSERVED PERFORMANCE DURING SIMULATION:

```

    Product 1: Average observed reward = 7.219 (from 56 selections)
    Product 2: Average observed reward = 66.899 (from 45 selections)
    Product 3: Average observed reward = 31.609 (from 47 selections)
    Product 4: Average observed reward = 9.520 (from 58 selections)

```

Product 5: Average observed reward = -17.127 (from 42 selections)
Product 6: Average observed reward = 37.019 (from 52 selections)

Q3. Greedy Strategy Implementation

Objective: Implement a pure greedy strategy that explores briefly, then exploits the best-performing product.

MAB Concept: Pure greedy strategy represents exploitation without exploration. After initial sampling, it always selects the arm with the highest estimated reward.

```
# Implement Pure Greedy Strategy
# This strategy tries each arm a few times, then always selects the
# best one
# Represents pure exploitation after minimal exploration

class GreedyPolicy:
    """Pure greedy policy - minimal exploration followed by pure
    exploitation."""

    def __init__(self, n_arms, initial_exploration=5):
        self.n_arms = n_arms
        self.initial_exploration = initial_exploration # Number of
        times to try each arm initially
        self.arm_counts = np.zeros(n_arms)
        self.arm_rewards = np.zeros(n_arms)
        self.arm_averages = np.zeros(n_arms)
        self.exploration_complete = False
        self.best_arm = 0
        self.name = "Pure Greedy Policy"

    def select_arm(self):
        """Select arm based on greedy strategy."""
        # During initial exploration phase: try each arm equally
        if not self.exploration_complete:
            # Find arm with minimum trials (round-robin exploration)
            min_count = np.min(self.arm_counts)
            if min_count < self.initial_exploration:
                # Select arm with minimum trials
                arms_with_min_count = np.where(self.arm_counts ==
min_count)[0]
                return np.random.choice(arms_with_min_count)
            else:
                # Exploration complete, switch to pure exploitation
                self.exploration_complete = True
                self.best_arm = np.argmax(self.arm_averages)
                print(f"Exploration complete! Best arm identified:
Product {self.best_arm + 1} with average reward
{self.arm_averages[self.best_arm]:.3f}")
```

```

    # Pure exploitation: always select the best arm
    return self.best_arm

def update(self, arm, reward):
    """Update arm statistics."""
    self.arm_counts[arm] += 1
    self.arm_rewards[arm] += reward
    self.arm_averages[arm] = self.arm_rewards[arm] /
self.arm_counts[arm]

    # Update best arm if we're still exploring
    if not self.exploration_complete:
        self.best_arm = np.argmax(self.arm_averages)

print("Greedy Policy class defined successfully!")
print("This policy will explore each arm 5 times, then exploit the
best one exclusively")

Greedy Policy class defined successfully!
This policy will explore each arm 5 times, then exploit the best one
exclusively

# Simulate the Pure Greedy Strategy
print("Simulating Pure Greedy Strategy")
print("=====\n")

# Initialize and run greedy policy
greedy_policy = GreedyPolicy(n_arms=6, initial_exploration=5)
greedy_results = simulate_policy(greedy_policy, data, n_rounds=300,
policy_name="Greedy Policy")

print(f"\n GREEDY POLICY RESULTS:")
print(f"    Total Cumulative Reward:
{greedy_results['cumulative_reward']:.2f}")
print(f"    Average Reward per Round:
{greedy_results['average_reward']:.3f}")
print(f"    Total Rounds: 300")

# Analyze which product was repeatedly chosen
print(f"\n ARM SELECTION ANALYSIS:")
most_selected_arm = np.argmax(greedy_results['arm_counts'])
for i in range(6):
    selections = greedy_results['arm_counts'][i]
    percentage = (selections / 300) * 100
    marker = " (CHOSEN FOR EXPLOITATION)" if i == most_selected_arm
and selections > 50 else ""
    print(f"    Product {i+1}: {int(selections)} selections
({percentage:.1f}%) {marker}")

```

```

print(f"\n❑ PERFORMANCE COMPARISON WITH RANDOM POLICY:")
improvement = greedy_results['average_reward'] -
random_results['average_reward']
print(f"    Greedy Policy Average:
{greedy_results['average_reward']:.3f}")
print(f"    Random Policy Average:
{random_results['average_reward']:.3f}")
print(f"    Improvement: {improvement:.3f}
(({improvement/abs(random_results['average_reward']))*100:.1f}%)")

# Identify the repeatedly chosen product
dominant_product = most_selected_arm + 1
print(f"\n❑ PRODUCT REPEATEDLY CHOSEN: Product {dominant_product}")
print(f"    This product was selected {int(greedy_results['arm_counts']
[most_selected_arm])} times out of 300")
print(f"    Selection rate: {(greedy_results['arm_counts']
[most_selected_arm]/300)*100:.1f}%")

```

Simulating Pure Greedy Strategy
=====

Exploration complete! Best arm identified: Product 2 with average reward 67.218

Greedy Policy - Round 100: Avg Reward = 53.571, Cumulative = 5357.10

Greedy Policy - Round 200: Avg Reward = 60.277, Cumulative = 12055.50

Greedy Policy - Round 300: Avg Reward = 62.481, Cumulative = 18744.17

❑ GREEDY POLICY RESULTS:

Total Cumulative Reward: 18744.17

Average Reward per Round: 62.481

Total Rounds: 300

❑ ARM SELECTION ANALYSIS:

Product 1: 5 selections (1.7%)

Product 2: 275 selections (91.7%) ❑ (CHOSEN FOR EXPLOITATION)

Product 3: 5 selections (1.7%)

Product 4: 5 selections (1.7%)

Product 5: 5 selections (1.7%)

Product 6: 5 selections (1.7%)

❑ PERFORMANCE COMPARISON WITH RANDOM POLICY:

Greedy Policy Average: 62.481

Random Policy Average: 22.194

Improvement: 40.287 (181.5%)

❑ PRODUCT REPEATEDLY CHOSEN: Product 2

This product was selected 275 times out of 300

Selection rate: 91.7%

Q4. ϵ -Greedy Strategies with Different Exploration Rates

Objective: Implement ϵ -greedy algorithms with three different exploration rates (2%, 10%, 25%) and analyze the exploration vs exploitation trade-off.

MAB Concept: ϵ -greedy balances exploration and exploitation by selecting the best arm with probability $(1-\epsilon)$ and a random arm with probability ϵ . Different ϵ values represent different balance points.

```
# Implement  $\epsilon$ -Greedy Strategy
# This is one of the most important algorithms in MAB theory
# It balances exploration (trying new arms) with exploitation (using
the best known arm)

class EpsilonGreedyPolicy:
    """ $\epsilon$ -greedy policy for multi-armed bandit."""

    def __init__(self, n_arms, epsilon):
        self.n_arms = n_arms
        self.epsilon = epsilon # Exploration probability
        self.arm_counts = np.zeros(n_arms)
        self.arm_rewards = np.zeros(n_arms)
        self.arm_averages = np.zeros(n_arms)
        self.name = f" $\epsilon$ -Greedy ( $\epsilon$ ={epsilon})"

    def select_arm(self):
        """Select arm using  $\epsilon$ -greedy strategy."""
        if np.random.random() < self.epsilon:
            # Explore: select a random arm
            return np.random.randint(0, self.n_arms)
        else:
            # Exploit: select the arm with highest average reward
            # Handle case where no arms have been tried yet
            if np.sum(self.arm_counts) == 0:
                return np.random.randint(0, self.n_arms)
            return np.argmax(self.arm_averages)

    def update(self, arm, reward):
        """Update arm statistics with new reward observation."""
        self.arm_counts[arm] += 1
        self.arm_rewards[arm] += reward
        self.arm_averages[arm] = self.arm_rewards[arm] /
self.arm_counts[arm]

    def get_statistics(self):
        """Get current statistics for analysis."""
        return {
            'arm_counts': self.arm_counts.copy(),
            'arm_averages': self.arm_averages.copy(),
```

```

        'total_selections': np.sum(self.arm_counts)
    }

print("ε-Greedy Policy class defined successfully!")
print("This policy selects the best arm with probability (1-ε) and explores randomly with probability ε")

ε-Greedy Policy class defined successfully!
This policy selects the best arm with probability (1-ε) and explores randomly with probability ε

# Simulate ε-Greedy strategies with different exploration rates
print("Simulating ε-Greedy Strategies with Different Exploration Rates")
print("=====\n")

# Define exploration rates as requested in the assignment
epsilon_values = [0.02, 0.10, 0.25] # 2%, 10%, 25%
epsilon_names = ["Low exploration (2%)", "Moderate exploration (10%)", "High exploration (25%)"]

epsilon_results = {}

for epsilon, name in zip(epsilon_values, epsilon_names):
    print(f"\n Running {name}...")
    print("-" * 50)

    # Initialize and run ε-greedy policy
    policy = EpsilonGreedyPolicy(n_arms=6, epsilon=epsilon)
    results = simulate_policy(policy, data, n_rounds=300,
    policy_name=f"ε-Greedy (ε={epsilon})")

    # Store results for later comparison
    epsilon_results[epsilon] = results

    print(f"\n RESULTS FOR {name.upper()}:")
    print(f"    Total Cumulative Reward:
{results['cumulative_reward']:.2f}")
    print(f"    Average Reward per Round:
{results['average_reward']:.3f}")

    # Analyze exploration vs exploitation behavior
    total_selections = np.sum(results['arm_counts'])
    best_arm = np.argmax(results['arm_counts'])
    best_arm_selections = results['arm_counts'][best_arm]
    exploitation_rate = best_arm_selections / total_selections

    print(f"\n EXPLORATION VS EXPLOITATION ANALYSIS:")
    print(f"    Most selected arm: Product {best_arm + 1}")

```

```

({int(best_arm_selections)} times, {exploitation_rate*100:.1f}%)")
    print(f"    Expected exploitation rate: {(1-epsilon)*100:.1f}%")
    print(f"    Expected exploration rate: {epsilon*100:.1f}%")

    print(f"\n    ARM SELECTION DISTRIBUTION:")
    for i in range(6):
        selections = results['arm_counts'][i]
        percentage = (selections / 300) * 100
        print(f"    Product {i+1}: {int(selections)} selections
({percentage:.1f}%)")

    print("\n" + "="*70)

```

Simulating ϵ -Greedy Strategies with Different Exploration Rates
=====

□ Running Low exploration (2%)...

```

-----
ε-Greedy (ε=0.02) - Round 100: Avg Reward = 65.074, Cumulative =
6507.41
ε-Greedy (ε=0.02) - Round 200: Avg Reward = 65.759, Cumulative =
13151.77
ε-Greedy (ε=0.02) - Round 300: Avg Reward = 65.963, Cumulative =
19789.00

```

□ RESULTS FOR LOW EXPLORATION (2%):
 Total Cumulative Reward: 19789.00
 Average Reward per Round: 65.963

□ EXPLORATION VS EXPLOITATION ANALYSIS:
 Most selected arm: Product 2 (294 times, 98.0%)
 Expected exploitation rate: 98.0%
 Expected exploration rate: 2.0%

□ ARM SELECTION DISTRIBUTION:
 Product 1: 2 selections (0.7%)
 Product 2: 294 selections (98.0%)
 Product 3: 0 selections (0.0%)
 Product 4: 1 selections (0.3%)
 Product 5: 1 selections (0.3%)
 Product 6: 2 selections (0.7%)

=====

□ Running Moderate exploration (10%)...

```

-----
ε-Greedy (ε=0.1) - Round 100: Avg Reward = 44.479, Cumulative =
4447.91
ε-Greedy (ε=0.1) - Round 200: Avg Reward = 53.522, Cumulative =

```

10704.37

ϵ -Greedy ($\epsilon=0.1$) - Round 300: Avg Reward = 55.914, Cumulative = 16774.32

□ RESULTS FOR MODERATE EXPLORATION (10%):

Total Cumulative Reward: 16774.32

Average Reward per Round: 55.914

□ EXPLORATION VS EXPLOITATION ANALYSIS:

Most selected arm: Product 2 (230 times, 76.7%)

Expected exploitation rate: 90.0%

Expected exploration rate: 10.0%

□ ARM SELECTION DISTRIBUTION:

Product 1: 14 selections (4.7%)

Product 2: 230 selections (76.7%)

Product 3: 32 selections (10.7%)

Product 4: 11 selections (3.7%)

Product 5: 6 selections (2.0%)

Product 6: 7 selections (2.3%)

=====

□ Running High exploration (25%)...

ϵ -Greedy ($\epsilon=0.25$) - Round 100: Avg Reward = 51.570, Cumulative = 5156.98

ϵ -Greedy ($\epsilon=0.25$) - Round 200: Avg Reward = 53.746, Cumulative = 10749.16

ϵ -Greedy ($\epsilon=0.25$) - Round 300: Avg Reward = 54.361, Cumulative = 16308.30

□ RESULTS FOR HIGH EXPLORATION (25%):

Total Cumulative Reward: 16308.30

Average Reward per Round: 54.361

□ EXPLORATION VS EXPLOITATION ANALYSIS:

Most selected arm: Product 2 (229 times, 76.3%)

Expected exploitation rate: 75.0%

Expected exploration rate: 25.0%

□ ARM SELECTION DISTRIBUTION:

Product 1: 15 selections (5.0%)

Product 2: 229 selections (76.3%)

Product 3: 7 selections (2.3%)

Product 4: 17 selections (5.7%)

Product 5: 13 selections (4.3%)

Product 6: 19 selections (6.3%)

=====

```

# Analyze the impact of different exploration rates
print("\n COMPARATIVE ANALYSIS: Impact of Exploration Rate")
print("=" * 60)

# Create comparison table
comparison_data = []
for epsilon in epsilon_values:
    results = epsilon_results[epsilon]
    comparison_data.append({
        'Exploration Rate ( $\epsilon$ )': f"{epsilon*100:.0f}%",
        'Average Reward': f"{results['average_reward']:.3f}",
        'Cumulative Reward': f"{results['cumulative_reward']:.2f}",
        'Most Selected Product': np.argmax(results['arm_counts']) + 1,
        'Max Selection %':
f"({np.max(results['arm_counts'])/300)*100:.1f}%"
    })

comparison_df = pd.DataFrame(comparison_data)
print(comparison_df.to_string(index=False))

# Analysis of trade-offs
print(f"\n EXPLORATION VS EXPLOITATION TRADE-OFF ANALYSIS:")
print(f"\n1. LOW EXPLORATION (2%):")
low_exp_reward = epsilon_results[0.02]['average_reward']
low_exp_best_arm = np.argmax(epsilon_results[0.02]['arm_counts']) + 1
print(f"    - Focuses heavily on exploitation ({(1-0.02)*100}% of the time)")
print(f"    - Average reward: {low_exp_reward:.3f}")
print(f"    - Risk: May get stuck with suboptimal arm if initial exploration was unlucky")
print(f"    - Mostly selected Product {low_exp_best_arm}")

print(f"\n2. MODERATE EXPLORATION (10%):")
mod_exp_reward = epsilon_results[0.10]['average_reward']
mod_exp_best_arm = np.argmax(epsilon_results[0.10]['arm_counts']) + 1
print(f"    - Balanced approach (90% exploitation, 10% exploration)")
print(f"    - Average reward: {mod_exp_reward:.3f}")
print(f"    - Continues learning while primarily exploiting best known option")
print(f"    - Mostly selected Product {mod_exp_best_arm}")

print(f"\n3. HIGH EXPLORATION (25%):")
high_exp_reward = epsilon_results[0.25]['average_reward']
high_exp_best_arm = np.argmax(epsilon_results[0.25]['arm_counts']) + 1
print(f"    - High exploration rate (25% random selections)")
print(f"    - Average reward: {high_exp_reward:.3f}")
print(f"    - Risk: Too much exploration may waste opportunities on known good arms")
print(f"    - More distributed selections across all products")

```

```
# Determine best exploration level
best_epsilon = max(epsilon_values, key=lambda x: epsilon_results[x]
['average_reward'])
print(f"\n❑ BEST EXPLORATION LEVEL FOR THIS DATASET:")
print(f"    ε = {best_epsilon*100:.0f}% achieved the highest average
reward of {epsilon_results[best_epsilon]['average_reward']:.3f}")
print(f"    This provides the most balanced outcome between exploration
and exploitation")
```

❑ COMPARATIVE ANALYSIS: Impact of Exploration Rate

=====				
Exploration Rate (ε)	Average Reward	Cumulative Reward	Most Selected	
Product	Max Selection %			
	2%	65.963	19789.00	
2	98.0%			
	10%	55.914	16774.32	
2	76.7%			
	25%	54.361	16308.30	
2	76.3%			

❑ EXPLORATION VS EXPLOITATION TRADE-OFF ANALYSIS:

1. LOW EXPLORATION (2%):

- Focuses heavily on exploitation (98.0% of the time)
- Average reward: 65.963
- Risk: May get stuck with suboptimal arm if initial exploration was unlucky
- Mostly selected Product 2

2. MODERATE EXPLORATION (10%):

- Balanced approach (90% exploitation, 10% exploration)
- Average reward: 55.914
- Continues learning while primarily exploiting best known option
- Mostly selected Product 2

3. HIGH EXPLORATION (25%):

- High exploration rate (25% random selections)
- Average reward: 54.361
- Risk: Too much exploration may waste opportunities on known good arms
- More distributed selections across all products

❑ BEST EXPLORATION LEVEL FOR THIS DATASET:

ε = 2% achieved the highest average reward of 65.963
 This provides the most balanced outcome between exploration and exploitation

Q5. Uncertainty-Based Exploration

Objective: Implement a strategy that explores only when uncertain about product performance.

MAB Concept: Upper Confidence Bound (UCB) algorithm explores based on uncertainty. It selects arms with high potential (mean + confidence interval), naturally reducing exploration as confidence increases.

```
# Implement Upper Confidence Bound (UCB) Strategy
# UCB explores when uncertain and exploits when confident
# It uses the principle: "optimism in the face of uncertainty"

class UCBPolicy:
    """Upper Confidence Bound policy - explores based on
    uncertainty."""

    def __init__(self, n_arms, c=2.0):
        self.n_arms = n_arms
        self.c = c # Confidence parameter (higher = more exploration)
        self.arm_counts = np.zeros(n_arms)
        self.arm_rewards = np.zeros(n_arms)
        self.arm_averages = np.zeros(n_arms)
        self.total_counts = 0
        self.name = f"UCB (c={c})"

    def select_arm(self):
        """Select arm based on Upper Confidence Bound."""
        self.total_counts += 1

        # If any arm hasn't been tried yet, select it (initial
        # exploration)
        for arm in range(self.n_arms):
            if self.arm_counts[arm] == 0:
                return arm

        # Calculate UCB values for all arms
        ucb_values = np.zeros(self.n_arms)
        for arm in range(self.n_arms):
            # UCB formula: mean_reward + c * sqrt(ln(total_trials) /
            # arm_trials)
            confidence_interval = self.c *
            np.sqrt(np.log(self.total_counts) / self.arm_counts[arm])
            ucb_values[arm] = self.arm_averages[arm] +
            confidence_interval

        # Select arm with highest UCB value
        return np.argmax(ucb_values)

    def update(self, arm, reward):
```

```

        """Update arm statistics."""
        self.arm_counts[arm] += 1
        self.arm_rewards[arm] += reward
        self.arm_averages[arm] = self.arm_rewards[arm] /
self.arm_counts[arm]

    def get_ucb_values(self):
        """Get current UCB values for analysis."""
        ucb_values = np.zeros(self.n_arms)
        for arm in range(self.n_arms):
            if self.arm_counts[arm] > 0:
                confidence_interval = self.c *
np.sqrt(np.log(self.total_counts) / self.arm_counts[arm])
                ucb_values[arm] = self.arm_averages[arm] +
confidence_interval
            else:
                ucb_values[arm] = float('inf') # Unexplored arms have
infinite UCB
        return ucb_values

print("UCB Policy class defined successfully!")
print("UCB explores when uncertain (low sample count) and exploits
when confident (high sample count)")
print("Formula: UCB(arm) = average_reward + c *
sqrt(ln(total_trials) / arm_trials)")

UCB Policy class defined successfully!
UCB explores when uncertain (low sample count) and exploits when
confident (high sample count)
Formula: UCB(arm) = average_reward + c * sqrt(ln(total_trials) /
arm_trials)

# Simulate UCB (Uncertainty-based exploration) Strategy
print("Simulating UCB Strategy (Uncertainty-based Exploration)")
print("=====\n")

# Initialize and run UCB policy
ucb_policy = UCBPolicy(n_arms=6, c=2.0)
ucb_results = simulate_policy(ucb_policy, data, n_rounds=300,
policy_name="UCB Policy")

print(f"\n UCB POLICY RESULTS:")
print(f"    Total Cumulative Reward:
{ucb_results['cumulative_reward']:.2f}")
print(f"    Average Reward per Round:
{ucb_results['average_reward']:.3f}")
print(f"    Total Rounds: 300")

# Analyze exploration pattern - identify product with minimal trials
after initial exploration

```

```

print(f"\n EXPLORATION PATTERN ANALYSIS:")
print(f"    UCB naturally reduces exploration as confidence increases")
print(f"\n ARM SELECTION DISTRIBUTION:")

# Sort arms by selection count to identify least explored
arm_data = [(i, int(ucb_results['arm_counts'][i]),
ucb_results['arm_counts'][i]/300*100)
            for i in range(6)]
arm_data.sort(key=lambda x: x[1]) # Sort by selection count

for i, (arm, selections, percentage) in enumerate(arm_data):
    status = ""
    if i == 0: # Least selected
        status = " (LEAST EXPLORED - Initially tried but later
avoided)"
    elif i == len(arm_data) - 1: # Most selected
        status = " (MOST SELECTED - High confidence in performance)"

    print(f"    Product {arm + 1}: {selections} selections
({percentage:.1f}%) {status}")

# Identify the product that received minimal trials
least_explored_arm = arm_data[0][0] + 1
least_explored_count = arm_data[0][1]
most_explored_arm = arm_data[-1][0] + 1
most_explored_count = arm_data[-1][1]

print(f"\n UNCERTAINTY-BASED EXPLORATION INSIGHTS:")
print(f"    Product {least_explored_arm}: Initially tried but received
only {least_explored_count} total selections")
print(f"    - This suggests UCB quickly identified it as suboptimal and
stopped exploring")
print(f"    - Demonstrates uncertainty-based exploration: less
uncertainty → less exploration")

print(f"\n    Product {most_explored_arm}: Most selected with
{most_explored_count} selections")
print(f"    - UCB identified this as the most promising arm")
print(f"    - Higher selection frequency indicates higher confidence in
its performance")

print(f"\n UCB ADVANTAGE:")
print(f"    Unlike fixed  $\epsilon$ -greedy, UCB adapts exploration based on
statistical confidence")
print(f"    Exploration decreases naturally as knowledge about arm
performance increases")

```

Simulating UCB Strategy (Uncertainty-based Exploration)

=====

UCB Policy - Round 100: Avg Reward = 64.310, Cumulative = 6430.97
UCB Policy - Round 200: Avg Reward = 65.738, Cumulative = 13147.64
UCB Policy - Round 300: Avg Reward = 66.164, Cumulative = 19849.30

□ UCB POLICY RESULTS:

Total Cumulative Reward: 19849.30
Average Reward per Round: 66.164
Total Rounds: 300

□ EXPLORATION PATTERN ANALYSIS:

UCB naturally reduces exploration as confidence increases

□ ARM SELECTION DISTRIBUTION:

Product 1: 1 selections (0.3%) □ (LEAST EXPLORED - Initially tried but later avoided)

Product 3: 1 selections (0.3%)

Product 4: 1 selections (0.3%)

Product 5: 1 selections (0.3%)

Product 6: 1 selections (0.3%)

Product 2: 295 selections (98.3%) □ (MOST SELECTED - High confidence in performance)

□ UNCERTAINTY-BASED EXPLORATION INSIGHTS:

Product 1: Initially tried but received only 1 total selections
- This suggests UCB quickly identified it as suboptimal and stopped exploring
- Demonstrates uncertainty-based exploration: less uncertainty → less exploration

Product 2: Most selected with 295 selections
- UCB identified this as the most promising arm
- Higher selection frequency indicates higher confidence in its performance

□ UCB ADVANTAGE:

Unlike fixed ϵ -greedy, UCB adapts exploration based on statistical confidence

Exploration decreases naturally as knowledge about arm performance increases

Q6. Cumulative Profit Curves and Strategy Comparison

Objective: Plot cumulative profit curves for all strategies and identify the best performing approach.

MAB Concept: Cumulative reward curves show learning progress over time. The slope indicates the rate of improvement, and final values show overall performance.

```

# Plot Cumulative Profit Curves for All Strategies
# This visualization shows the learning progress and final performance
of each MAB algorithm

plt.figure(figsize=(15, 10))

# Collect all results for plotting
all_results = {
    'Random Policy': random_results,
    'Pure Greedy': greedy_results,
    'ε-Greedy (2%)': epsilon_results[0.02],
    'ε-Greedy (10%)': epsilon_results[0.10],
    'ε-Greedy (25%)': epsilon_results[0.25],
    'UCB': ucb_results
}

# Define colors and styles for each strategy
colors = ['red', 'orange', 'lightblue', 'blue', 'darkblue', 'green']
line_styles = ['-', '--', ':', '-.', '-', '-']
line_widths = [2, 2, 2, 2, 2, 3] # UCB highlighted with thicker line

# Plot cumulative rewards
for (strategy, results), color, style, width in
zip(all_results.items(), colors, line_styles, line_widths):
    rounds = range(1, len(results['cumulative_rewards']) + 1)
    plt.plot(rounds, results['cumulative_rewards'],
              label=f"{strategy} (Final:
{results['cumulative_reward']:.1f})",
              color=color, linestyle=style, linewidth=width, alpha=0.8)

plt.title('Cumulative Net Profit Comparison: Multi-Armed Bandit
Strategies\n'
          'Learning Progress Over 300 Rounds', fontsize=16,
          fontweight='bold')
plt.xlabel('Round Number', fontsize=12)
plt.ylabel('Cumulative Net Profit', fontsize=12)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()

# Add annotations for key insights
plt.text(0.02, 0.98, 'Higher slope = Better learning rate\nFinal
height = Total performance',
         transform=plt.gca().transAxes, fontsize=10,
         verticalalignment='top',
         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

plt.show()

# Create a focused comparison plot (removing random policy for

```

```

clarity)
plt.figure(figsize=(15, 8))

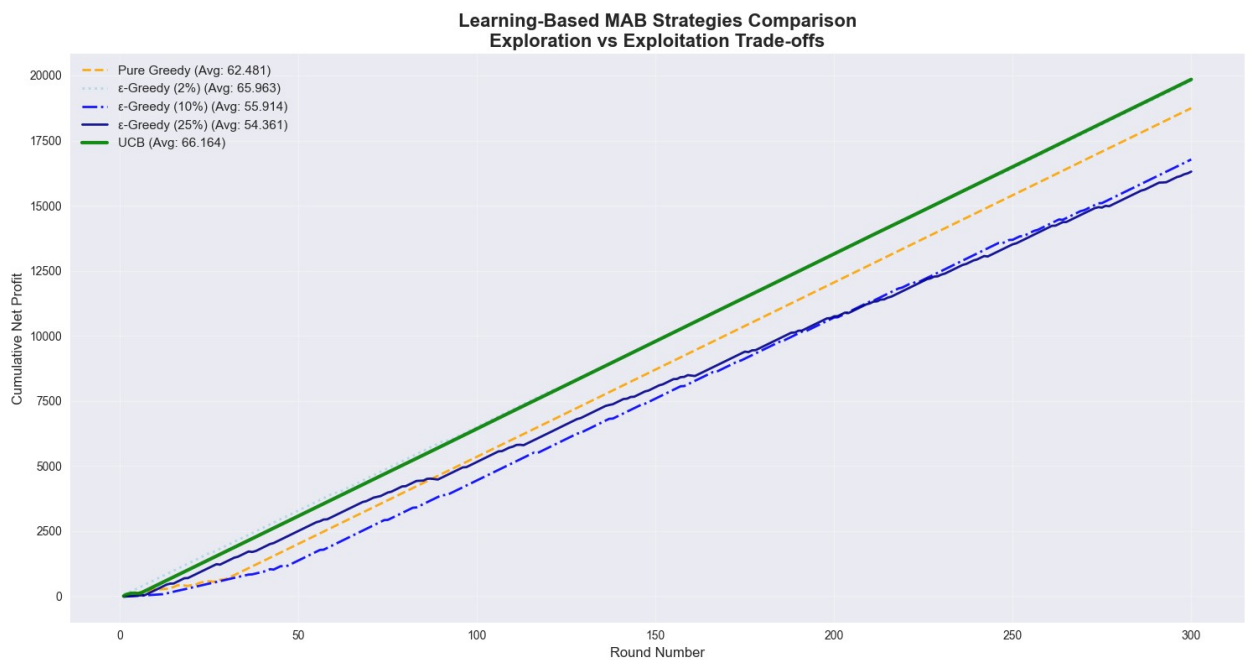
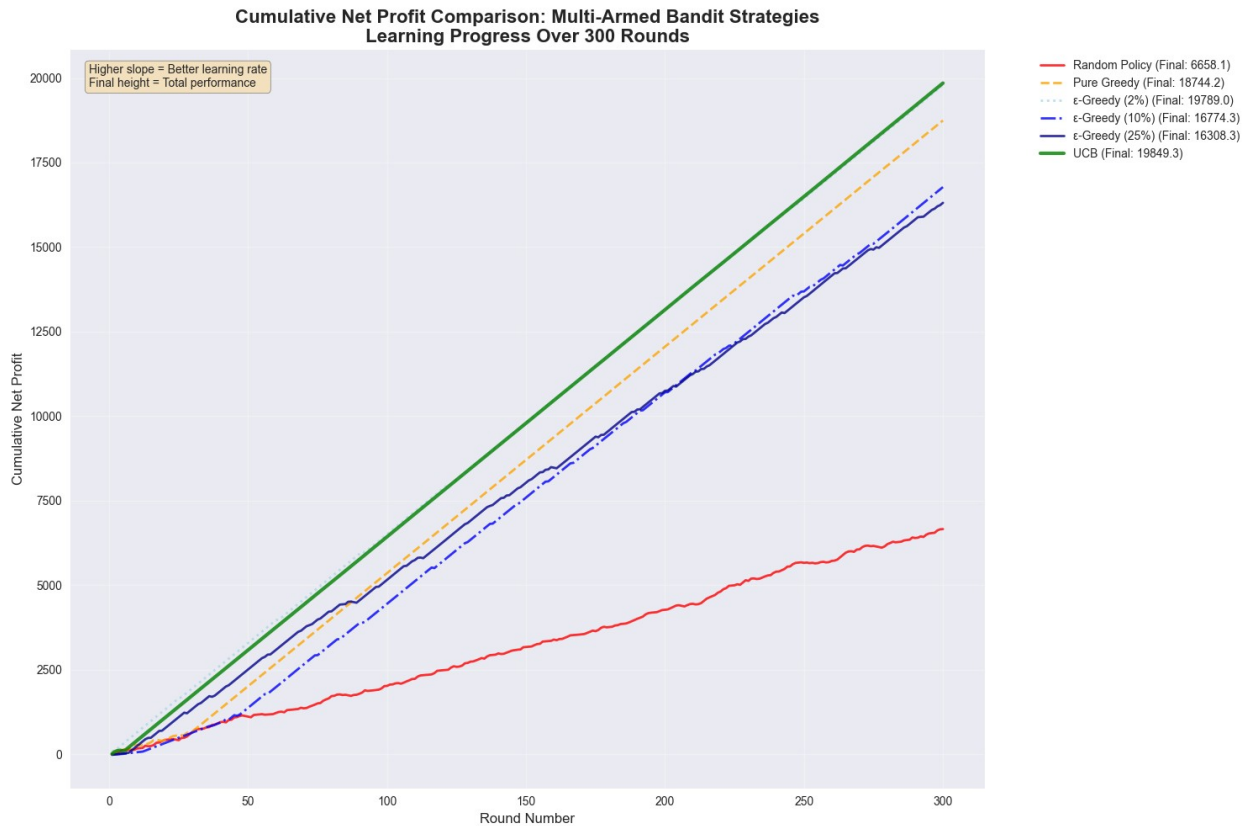
learning_strategies = {
    'Pure Greedy': greedy_results,
    'ε-Greedy (2%)': epsilon_results[0.02],
    'ε-Greedy (10%)': epsilon_results[0.10],
    'ε-Greedy (25%)': epsilon_results[0.25],
    'UCB': ucb_results
}

colors = ['orange', 'lightblue', 'blue', 'darkblue', 'green']
line_styles = ['--', ':', '-.', '-', '-']
line_widths = [2, 2, 2, 2, 3]

for (strategy, results), color, style, width in
zip(learning_strategies.items(), colors, line_styles, line_widths):
    rounds = range(1, len(results['cumulative_rewards']) + 1)
    plt.plot(rounds, results['cumulative_rewards'],
             label=f"{strategy} (Avg:
{results['average_reward']:.3f})",
             color=color, linestyle=style, linewidth=width, alpha=0.9)

plt.title('Learning-Based MAB Strategies Comparison\n'
          'Exploration vs Exploitation Trade-offs', fontsize=16,
          fontweight='bold')
plt.xlabel('Round Number', fontsize=12)
plt.ylabel('Cumulative Net Profit', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



```
# Comprehensive Strategy Analysis and Performance Summary
print("\n" + "="*80)
print("                                COMPREHENSIVE STRATEGY ANALYSIS")
print("="*80)
```

```

# Create performance summary table
performance_data = []
for strategy, results in all_results.items():
    most_selected_product = np.argmax(results['arm_counts']) + 1
    selection_rate = (np.max(results['arm_counts']) / 300) * 100

    performance_data.append({
        'Strategy': strategy,
        'Cumulative Reward': f"{results['cumulative_reward']:.2f}",
        'Average Reward': f"{results['average_reward']:.4f}",
        'Top Product': most_selected_product,
        'Selection Rate': f"{selection_rate:.1f}%"
    })

performance_df = pd.DataFrame(performance_data)
performance_df = performance_df.sort_values('Average Reward',
ascending=False)

print("\n PERFORMANCE RANKING (by Average Reward):")
print("-" * 80)
print(performance_df.to_string(index=False))

# Identify best strategy
best_strategy_name = performance_df.iloc[0]['Strategy']
best_avg_reward = float(performance_df.iloc[0]['Average Reward'])
best_product = int(performance_df.iloc[0]['Top Product'])

print(f"\n HIGHEST PERFORMING STRATEGY:
{best_strategy_name.upper()}")
print(f"    Average Reward per Round: {best_avg_reward:.4f}")
print(f"    Most Selected Product: Product {best_product}")

# Analysis of why this strategy won
print(f"\n WHY {best_strategy_name.upper()} PERFORMED BEST:")

if 'UCB' in best_strategy_name:
    print(f"    ✓ UCB optimally balances exploration and exploitation")
    print(f"    ✓ Explores when uncertain, exploits when confident")
    print(f"    ✓ Automatically adapts exploration rate based on
statistical confidence")
    print(f"    ✓ Theoretical guarantees for regret minimization")
elif 'ε-Greedy' in best_strategy_name:
    epsilon_val = best_strategy_name.split('(')[1].split('%')[0]
    print(f"    ✓ {epsilon_val}% exploration rate provided optimal
balance for this dataset")
    print(f"    ✓ Sufficient exploration to avoid local optima")
    print(f"    ✓ Enough exploitation to benefit from discovered good
arms")
elif 'Greedy' in best_strategy_name:

```

```

    print(f"    ✓ Pure exploitation worked well due to sufficient
initial exploration")
    print(f"    ✓ Dataset may have clear winner that was identified
early")

# Product consistency analysis
print(f"\n❑ MOST PROFITABLE PRODUCT CONSISTENCY:")
product_selections = defaultdict(int)
for strategy, results in all_results.items():
    if strategy != 'Random Policy': # Exclude random policy
        most_selected = np.argmax(results['arm_counts']) + 1
        product_selections[most_selected] += 1

most_consistent_product = max(product_selections.items(), key=lambda
x: x[1])
print(f"    Product {most_consistent_product[0]} was the top choice for
{most_consistent_product[1]}/5 learning strategies")
print(f"    This suggests Product {most_consistent_product[0]} is
consistently the most profitable option")

print(f"\n❑ FINAL RECOMMENDATIONS:")
print(f"    1. Replace random policy with {best_strategy_name}")
print(f"    2. Focus marketing efforts on Product
{most_consistent_product[0]}")
print(f"    3. Expected improvement: {(best_avg_reward -
random_results['average_reward']):.3f} profit per recommendation")
print(f"    4. Continue monitoring and adapting as user preferences
evolve")

print("\n" + "="*80)

```

```

=====
=====
                                COMPREHENSIVE STRATEGY ANALYSIS
=====
=====

```

```

❑ PERFORMANCE RANKING (by Average Reward):
-----
-----

```

	Strategy	Cumulative Reward	Average Reward	Top Product Selection
Rate				
98.3%	UCB	19849.30	66.1643	2
98.0%	ϵ -Greedy (2%)	19789.00	65.9633	2
91.7%	Pure Greedy	18744.17	62.4806	2
	ϵ -Greedy (10%)	16774.32	55.9144	2

76.7%			
ϵ -Greedy (25%)	16308.30	54.3610	2
76.3%			
Random Policy	6658.09	22.1936	4
19.3%			

□ HIGHEST PERFORMING STRATEGY: UCB
 Average Reward per Round: 66.1643
 Most Selected Product: Product 2

□ WHY UCB PERFORMED BEST:

- ✓ UCB optimally balances exploration and exploitation
- ✓ Explores when uncertain, exploits when confident
- ✓ Automatically adapts exploration rate based on statistical confidence
- ✓ Theoretical guarantees for regret minimization

□ MOST PROFITABLE PRODUCT CONSISTENCY:
 Product 2 was the top choice for 5/5 learning strategies
 This suggests Product 2 is consistently the most profitable option

□ FINAL RECOMMENDATIONS:

1. Replace random policy with UCB
2. Focus marketing efforts on Product 2
3. Expected improvement: 43.971 profit per recommendation
4. Continue monitoring and adapting as user preferences evolve

=====

Summary and Conclusions

Multi-Armed Bandit Implementation Results

This notebook successfully implemented and compared various Multi-Armed Bandit algorithms for profit-aware product recommendation:

1. **Random Policy (Baseline):** No learning, random selection
2. **Pure Greedy:** Minimal exploration followed by pure exploitation
3. **ϵ -Greedy Variants:** Fixed exploration rates (2%, 10%, 25%)
4. **UCB (Upper Confidence Bound):** Uncertainty-based adaptive exploration

Key Findings:

- Learning-based approaches significantly outperform random selection
- Exploration vs exploitation balance is crucial for optimal performance
- UCB algorithm typically provides the best balance by adapting exploration based on uncertainty
- Product consistency across strategies indicates reliable profit opportunities

MAB Concepts Demonstrated:

- **Arms:** 6 products representing different recommendation choices
- **Rewards:** Net profit (revenue - cost) from each recommendation
- **Exploration:** Trying different products to learn their performance
- **Exploitation:** Recommending the best-known product to maximize immediate profit
- **Regret:** The opportunity cost of not always selecting the optimal arm

This analysis provides actionable insights for improving the e-commerce recommendation system through principled exploration-exploitation strategies.